DESENVOLVIMENTO MOBILE

ALEX ZACHARIAS

ALEX@EINSTEINLIMEIRA.COM.BR

INTERPOLAÇÃO DE STRINGS

- Ao montar um texto mesclando strings com variáveis, podemos fazer igual a linha abaixo:

```
println("A soma de " + valor1 + " com " + valor2 + " é: " + total)
```

- Mas o Kotlin prefere que utilizemos a interpolação onde colocamos as variáveis dentro da própria string utilizando $ antes da variável para indicar para ele que ali está uma variável, neste caso o exemplo acima ficaria assim:

```
println("A soma de $valor1 com $valor2 é: $total")
```

- Para passar uma função na interpolação, devemos utilizar ${função()}

```
println("A soma de $valor1 com $valor2 é: ${soma( num1: 9, num2: 8 )}" )
```

FOR – ESTRUTURA DE REPETIÇÃO

- O comando for do Kotlin utiliza a seguinte sintaxe:

```
for( [item] in [coleção] step [passo] ) {
    [aqui vai seu código]
}
```

- Onde:
  - Item é sua variável
  - Coleção é a faixa de valores que queremos utilizar
  - Passo é onde definimos de quantos em quantos números é para ele andar, esse parâmetro é opcional se ele não for definido o passo padrão é 1

FOR – ESTRUTURA DE REPETIÇÃO

```
fun main() {
    for ( i in 0 ≤ .. ≤ 10 ){
        println( i )
    }
}
```

- O exemplo acima vai de 0 até 10 andando de 1 em 1 colocando os valores na variável i
- Os dois símbolos de menor ou igual não fomos nós que digitamos, é o IntelliJ que mostra para facilitar o entendimento do código. Digitamos apenas 0..10

FOR – ESTRUTURA DE REPETIÇÃO

```
fun main() {
    for ( i in 1 ≤ .. ≤ 20 step 2 ){
        println( i )
    }
}
```

- Aqui estamos indo de 1 até 20 mas com o passo de 2 em 2

```
C:\Users\alex\.jdks\openjdk-20.0.2\bin
1
3
5
7
9
11
13
15
17
19
```

FOR – ESTRUTURA DE REPETIÇÃO

- Podemos utilizar o for de forma decrescente, utilizando o comando downTo

```
fun main() {
    for ( i in 10 ≥ downTo ≥ 0 ){
        println( i )
    }
}
```

- Note que para fazer de forma decrescente não utilizamos ..
- Aqui também o maior ou igual foi incluído pelo IntelliJ

FOR – ESTRUTURA DE REPETIÇÃO

- Na sequencia decrescente também podemos utilizar o step para alterar o passo dele:

```
fun main() {
    for ( i in 15 ≥ downTo ≥ 0 step 3 ){
        println( i )
    }
}
```

Resultado:

```
C:\Users\alex\.jdk
15
12
9
6
3
0
```

FOR – ESTRUTURA DE REPETIÇÃO

- O for também pode ser utilizado para pegar cada um dos caracteres de uma string:

```
fun main() {
    val texto = "Einstein"
    for ( ch in texto ){
        println( ch )
    }
}
```

```
C:\Users\alex\.jdks\openjdk-2
E
i
n
s
t
e
i
n
```

WHILE – ESTRUTURA DE REPETIÇÃO

- O comando while do Kotlin utiliza a seguinte sintaxe:

```
while( [teste lógico] ) {
    [aqui vai seu código]
}
```

- Onde:
  - Teste lógico é onde você deve fazer a comparação que vai fazer com que o programa fique dentro do while enquanto ela for verdadeira.
  - Se ao chegar no comando while, seu teste lógico já for falso, o programa não executará o código dentro do while nenhuma vez.

WHILE – ESTRUTURA DE REPETIÇÃO

```
fun main() {
    print( "Digite um valor: ")
    val a = readLine()
    var b = 0
    if ( a != null && a != "")
        b = a.toInt()

    while ( b < 10 ){
        print( "$b " )
        b++
    }
}
```

WHILE – ESTRUTURA DE REPETIÇÃO

- Entendendo o código:

```
print( "Digite um valor: ")
val a = readLine()
```

- Pede para digitar um valor e grava na variável a

```
var b = 0
if ( a != null && a != "")
    b = a.toInt()
```

- Se a não for nulo nem vazio, transforma ele em inteiro e coloca o valor em b, caso contrário b continua como o valor 0 que ele foi declarado.
- Não estamos tratando quando o usuário digitar texto, nesse caso o programa irá dar um erro.

WHILE – ESTRUTURA DE REPETIÇÃO

```
while ( b < 10 ){
    print( "$b " )
    b++
}
```

- Imprime o valor de b na tela e soma 1 em b enquanto seu valor for menor que 10
- Se o valor digitado para b for maior ou igual a 10, ele não entra nenhuma vez dentro do while.

DO WHILE – ESTRUTURA DE REPETIÇÃO

- O comando do while do Kotlin utiliza a seguinte sintaxe:

```
do {
    [aqui vai seu código]
}
while( [teste lógico] )
```

- Onde:
  - Teste lógico é onde você deve fazer a comparação que vai fazer com que o programa execute novamente o código do do while enquanto esta condição for verdadeira ela for verdadeira.

DO WHILE – ESTRUTURA DE REPETIÇÃO

- Diferença entre o while e o do while:
- while só executa o código dentro dele se a condição for verdadeira.
- do while executa o código uma vez e só depois checa se a condição é verdadeira.
- **Portanto o do while sempre executa uma vez o código, mesmo que a condição seja falsa.**

DO WHILE – COMPARAÇÃO COM O WHILE

```
fun main() {
    var b = 10
    do {
        print( "$b " )
        b++
    }
    while ( b < 10 )
}
```

```
C:\Users\alex\.jdks\ope
10
Process finished with e
```

```
fun main() {
    var b = 10
    while ( b < 10 ){
        print( "$b " )
        b++
    }
}
```

```
C:\Users\alex\.jdks\open

Process finished with ex
```

FUNÇÕES

- Para declarar uma função utilizamos a palavra fun
- Uma função pode ser declarada antes ou depois da função main

```
fun main() {
    olaMundo()
}

fun olaMundo(){
    println( "Ola mundo!")
}
```

```
fun olaMundo(){
    println( "Ola mundo!")
}

fun main() {
    olaMundo()
}
```

FUNÇÕES

- Uma função pode ou não receber parâmetros, se for receber parâmetros devemos declarar o tipo do mesmo:

```
fun main() {
    print("Digite o primeiro valor: ")
    val valor1 = readln()
    print("Digite o segundo valor: ")
    val valor2 = readln()
    soma( valor1.toInt(), valor2.toInt() )
}

fun soma( num1: Int, num2: Int) {
    val total = num1 + num2
    println("A soma de $num1 com $num2 é: $total" )
}
```

- A função soma esta recebendo dois parâmetros do tipo inteiro

FUNÇÕES

- Uma função também pode retornar um valor, para isso devemos declarar o tipo de valor que será retornado no final da declaração da mesma.
- Toda função que retorna algo precisa ter o comando return dentro dela:

```
fun soma( num1: Int, num2: Int): Int {
    val total = num1 + num2
    return total
}
```

FUNÇÕES EM UM LINHA

- A função do exemplo anterior pode ser simplificada, eliminando a variável total:

```
fun soma(num1: Int, num2: Int): Int {
    return num1 + num2
}
```

- Essa função é tão simples que podemos fazer a mesma em uma única linha:

```
fun soma(num1: Int, num2: Int): Int = num1 + num2
```

FUNÇÕES - SOBRECARGA

- Se precisarmos declarar uma mesma função, mas recebendo parâmetros de tipos diferentes ou quantidade diferentes de parâmetros, podemos utilizar a sobrecarga.
- Basta declarar as funções com o mesmo nome alterando apenas os parâmetros ou o retorno das mesmas:

```
fun soma(num1: Int, num2: Int): Int = num1 + num2

fun soma(num1: Double, num2: Double): Double = num1 + num2
```

FUNÇÕES – PARÂMETROS COM VALOR PADRÃO

- Podemos omitir um parâmetro da função se o mesmo for definido com um valor padrão.
- Se passarmos o parâmetro o valor informado substitui o valor padrão.

```
fun main() {
   println( soma( num1: 10, num2: 12 ) )
   println( soma( num1: 10, num2: 12, num3: 15 ) )
}

fun soma(num1: Int, num2: Int, num3: Int = 0): Int {
    val total = num1 + num2 + num3
    return total
}
```

FUNÇÕES – NOMEANDO PARÂMETROS

- O Kotlin permite que os parâmetros de uma função seja passado fora de ordem, bastando apenas nomear os mesmos:

```
fun main() {
    media( disciplina: "Matemática", nota1: 7.0, nota2: 8.5)
    media(  nota1 = 9.5, disciplina = "Portugues", nota2 = 8.0 )
}

fun media( disciplina: String, nota1: Double, nota2: Double) {
    val media = (nota1 + nota2) / 2
    println( "a média final de $disciplina é $media")

}
```

FUNÇÕES – VARARG

- Podemos criar funções que recebem um numero indeterminado de parâmetros, pra isso utilizamos o vararg
- O vararq não é um tipo de dados, sendo necessário informar também o tipo de dado da variável que estamos criando.

```
fun media( vararg notas: Float): Float {
```

- No exemplo acima a função media recebe uma quantidade indefinida de parâmetros do tipo Float
- Alguns tipos de dados não aceitam o vararg, como o Int e o double por exemplo.

FUNÇÕES – VARARG

- Para utilizar os parâmetro recebidos pelo vararg podemos utiliza o comando for fazendo com que ele nos forneça um de cada vez:

```
fun media( vararg notas: Float): Float {
    var soma = 0f

    for (nota in notas){
        soma += nota
    }
    return (soma / notas.size)
}
```

FUNÇÕES – VARARG

```
fun media( vararg notas: Float): Float {
```

- Recebemos um parâmetro contendo uma quantidade indeterminada de notas

```
for (nota in notas){
```

- Utilizamos o for para pegar na variável nota cada uma das notas recebidas no parâmetro notas

```
soma += nota
```

- Vamos somando cada uma das notas na variável soma

```
return (soma / notas.size)
```

- Retornamos a média, o comando size de uma variável vararg retorna a quantidade de elementos dessa variável.

FUNÇÕES MATEMÁTICAS

- Vamos ver algumas funções matemáticas que fazem parte da biblioteca do Kotlin.
- Função max() retorna o maior entre dois valores

```
import kotlin.math.max

fun main() {
    println( max( a: 5,  b: 12))
}
```

- Note na primeira linha que foi importada a biblioteca math do Kotlin, especificamente a função max da biblioteca

FUNÇÕES MATEMÁTICAS

- Função min() retorna o menor de dois valores:

```
import kotlin.math.max
import kotlin.math.min

fun main() {
    println( max( a: 5, b: 12))
    println( min( a: 5, b: 12))
}
```

- Também foi importada da biblioteca math função min

FUNÇÕES MATEMÁTICAS

- Função sqrt() retorna a raiz quadrada de um numero:

```
import kotlin.math.*

fun main() {
    println( max( a: 5, b: 12 ))
    println( min( a: 5, b: 12 ))
    println( sqrt( x: 81.0 ))
}
```

- A função sqrt() recebe um numero double, para passar um inteiro temos que colocar a casa decimal como foi feito no exemplo acima.
- Quando utilizamos varias funções da mesma biblioteca, podemos colocar um * para que ele importe todas as funções de uma vez.

FUNÇÕES MATEMÁTICAS

- Função round() arredonda um valor.
- Essa função recebe um numero com casas decimais, não recebe inteiro.

```
import kotlin.math.*

fun main() {
    println( round( x: 345.7654 ))
}
```

- Constante PI retorna o valor de PI:

```
1  import kotlin.math.*
2
3  fun main() {
4      println( PI )
5  }
```

```
Run    FuncaoKt
C:\Users\alex\.jdks\openjdk-20.0
3.141592653589793
```

FUNÇÕES MATEMÁTICAS

- Função pow() retorna a potencia do numero que esta na variável.
- Recebe double ou float e retorna o mesmo tipo que recebeu:

```
import kotlin.math.*

fun main() {
    val x = 2.0
    val y = x.pow( n: 4)
    println( y )
}
```

- O exemplo acima eleva 2 a 4ª potencia